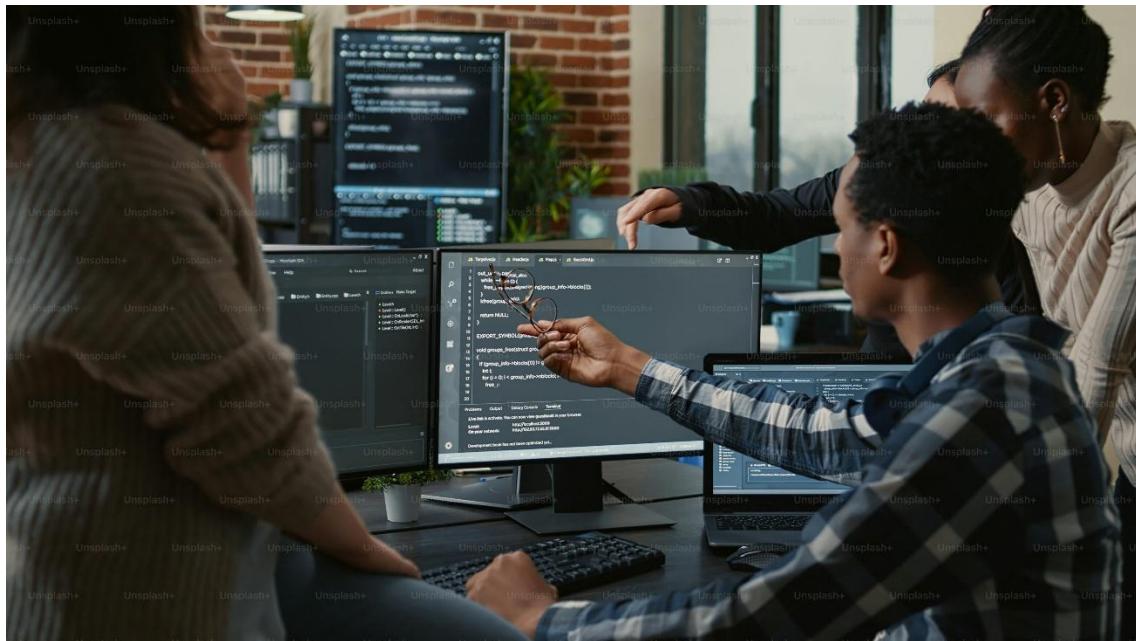


INDIVIDUAL REPORT

Team Project: Refactoring and Improving the Clinic Appointment & Patient Records API



Team Member : Maryam Malik AL-Muharrami

Date :26/6/2026

Project Overview

The assigned project was a Spring Boot REST API for managing doctors, patients, appointment slots, appointments, and visit records. The original system already supported the main clinic workflow, including booking, cancellation without deleting appointment history, rescheduling, and visit recording.

During the team project, we reviewed the existing code together, identified areas for improvement, divided the tasks, integrated our changes, and tested the final system. The objective was to improve the project without changing its main business rules.

Main Improvements Made by the Team

The team improved the project in several areas. We strengthened validation for request data, standardized doctor specialties using an enum, added clearer API response DTOs, improved error handling, added doctor search support, and improved the README documentation and repository organization.

We also improved the list endpoints by adding pagination, so doctors and patients can be returned in smaller pages instead of loading all records at once. The project documentation was updated to explain the API behaviour, validation rules, and main endpoints more clearly.

My Individual Contribution

My main tasks were Task 5: Add Phone Number Validation and Task 9: Implement Pagination for List Endpoints.

For phone number validation, I added a rule to ensure that patient phone numbers contain exactly eight digits and start with 9. Invalid numbers are now rejected with a clear HTTP 400 Bad Request message. This improves data quality and prevents incorrect contact information from being saved.

For pagination, I updated the doctor and patient list endpoints to accept page and size parameters. This allows the API to return records in organized pages and makes the system more scalable when the number of records increases. I also added validation for invalid page values, such as negative page numbers or a page size of zero.

Challenges Faced

One challenge was understanding how changes in one layer could affect other parts of the system. The project uses controllers, services, repositories, entities, DTOs, validation annotations, and exception handling. Therefore, every change needed to be checked carefully.

Another challenge was integrating the work of all team members. Since different tasks affected related parts of the project, we discussed the expected API behaviour and tested the system after merging our changes. We also made sure that important business rules, such as cancellation without deletion and slot availability updates, continued to work correctly.

Issues Identified and How They Were Resolved

The original project needed stronger input validation, clearer error messages, and better handling of larger lists of data. The team solved these issues by adding validation rules, improving the `GlobalExceptionHandler`, using DTOs for clearer responses, and adding pagination and search support.

A specific issue I handled was that a phone number could be non-empty but still invalid. This was resolved using a validation pattern. Another issue was that list endpoints returned all records at once. This was resolved by using Spring Data pagination with `Page`, `Pageable`, and `PageRequest`.

Technical Decisions

The team kept the existing layered Spring Boot architecture because it was suitable for the project. Controllers handle HTTP requests, services apply business rules, repositories access the H2 database, and entities represent database tables.

We used validation annotations to reject invalid input early, DTOs to avoid exposing internal entity details, enums to keep doctor specialties consistent, and pagination to improve scalability. We also kept cancellation and rescheduling as status changes instead of deleting appointment records, because this preserves appointment history.

Suggestions for Future Improvement

Future improvements could include automated tests using JUnit and MockMvc, pagination with DTO responses for all list endpoints, patient search filters, a maximum limit for page size, and authentication with different roles for administrators, reception staff, and doctors.

Lessons Learned

This project showed me that a working application can still need important improvements in validation, API design, error handling, documentation, and scalability. I also learned that team integration is as important as completing individual tasks. All changes must work together and must be tested after merging.

Conclusion

The final version of the Clinic API is more reliable, easier to use, and more prepared for future growth. My contribution focused on phone number validation and pagination, while the final result was achieved through teamwork, integration, testing, and shared review of the complete system.